ASSIGNMENT 3-QUESTIONS

CO3: Examine cryptographic hash algorithms and digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication and non-repudiation.

Q1 – What Makes a Hash Function Trustworthy? (6 Marks)
Part A – Avalanche and Fixed-Length Digests
Shared Group Work Together:

- Understand the required properties of a cryptographic hash (pre-image, second pre-image and collision resistance, avalanche effect)
- Implement a deliberately weak toy hash (for example, a byte-sum or XOR-fold hash) alongside a standard hash (SHA-256 from a library, used for comparison only)
- Decide how "avalanche" will be measured
- Run and test the experiment

Student A – Design & Mathematical Analysis
Student A must individually:

- Explain what each property means as an attack scenario
- Derive the expected number of output bits that change when one input bit is flipped in an ideal hash
- Explain mathematically why the toy hash fails the avalanche and second pre-image properties
- Explain why a fixed-length digest must have collisions

Student B – Experimental Validation
Student B must individually:

- Design the experimental inputs (single-bit flips, similar messages, long and short messages)
- Instrument the code to count the number of changed output bits
- Test at least three message sizes and a minimum number of trials per size
- Compare predicted and observed bit-change averages
- Explain any discrepancy

Part B – Collision Effort and the Birthday Bound
Shared Group Work Together, build a collision finder for a truncated hash (take only the first k bits of SHA-256, for k = 16, 20, 24, 28, 32).
Student A – Experimental Investigation
Student A must:

- Design the collision-search runs and repeat each several times
- Record the number of hashes computed before the first collision
- Investigate increasing values of k
- Present the experimental evidence in a table and graph

Student B – Birthday-Bound Reasoning

Student B must:

- Derive why the expected effort is roughly 2^(k/2), not 2^k
- Explain the difference between finding any collision and finding a collision with a specific message
- Explain how this determines the minimum safe digest length
- Relate the result to the retirement of MD5 and SHA-1

Individual Verification

Faculty may ask Student A to explain the birthday bound and Student B to explain the avalanche experiment. Primary responsibility does not mean exclusive knowledge.

Q2 – Integrity Is Not the Same as Authentication: Hash, MAC and HMAC (7 Marks)

Problem Context

A software distributor publishes a file together with its SHA-256 digest on the same web page. Another company protects internal messages with the construction H(key || message).

Shared Group Work

The pair must:

- Implement a simple Merkle–Damgård style toy hash (fixed block size, chaining value, padding with length)
- Implement the naive MAC H(key || message) and the HMAC construction
- Design tamper scenarios (modified message, modified message and recomputed digest, appended data)
- Conduct the experiments

Student A – Attack and Construction Investigation

Student A is individually responsible for:

- Showing that an attacker who can change both the file and the published digest defeats a plain hash
- Demonstrating a length-extension attack on the naive H(key || message) using your toy Merkle–Damgård hash
- Recording what the attacker needs to know and what the attacker does not need to know
- Explaining which property of the chaining construction makes the attack possible

Student B – Comparative Security Analysis

Student B is individually responsible for:

Explaining the structure of HMAC (inner and outer hash, key padding) and why it resists length extension

- Comparing hash-only, naive MAC and HMAC in a table: which provides integrity, which provides authentication, and which provides neither
- Explaining why a MAC cannot provide non-repudiation
- Verifying through tests that HMAC detects every tamper scenario you designed Joint Design Question

Together answer:

If you were designing the download-verification and message-protection components for an organisation, which mechanism would you choose for each, what must be kept secret, and what residual risk remains?

Both students must be able to defend the answer.

Q3 – Digital Signatures: Hash-then-Sign and What Can Break Them (7 Marks)

Shared Group Work

Together:

- Implement RSA signature generation and verification using hash-then-sign (small keys for measurement)
- Implement a DSA-style (or ECDSA-style) signature using toy parameters
- Test valid signatures, modified messages, modified signatures and wrong public keys
- Conduct the nonce experiment described below

Student A – Algorithmic Derivation

Student A must individually:

- Explain why the message is hashed before signing and what would go wrong with signing the raw message
- Prove why verification succeeds for a correctly generated RSA signature ($s = H(m)^d \bmod n$ and $s^e \equiv H(m) \bmod n$)
- Derive the DSA/ECDSA verification equation and show why it recovers the signer's value
- Explain what security goal each step contributes (integrity, authentication, non-repudiation)

Student B – Attack Experiment and Evidence

Student B must individually:

- Design the test messages and tamper cases and record accept/reject results
- Demonstrate nonce reuse in DSA/ECDSA: sign two messages with the same per-signature secret k, then recover k and the private key from the two signatures
- Demonstrate a textbook (no hash) RSA signature forgery (for example, multiplicative property: signature of $m_1 \cdot m_2$ from signatures of m_1 and m_2)
- Explain what these results show about "correct maths but unsafe implementation"

Joint Conclusion

Together answer:

A valid signature proves that someone holding the private key signed the message. Under what conditions does this actually amount to non-repudiation, and what could the signer claim to deny it?

Consider key compromise, timestamping, and the role of a trusted third party.

Q4 – Trust Through Certificates: X.509 Chain Validation (7 Marks)

Here the responsibilities are deliberately reversed so that the same student does not always perform the theoretical component.

Shared Group Work

Together:

- Study the structure of an X.509 v3 certificate (subject, issuer, serial number, validity, public key, extensions such as key usage and basic constraints, signature)
- Build a small three-level hierarchy (root CA → intermediate CA → end-entity) using a library or a toolkit such as OpenSSL
- Implement a chain-validation procedure that follows the steps a client must perform
- Verify the procedure on valid and invalid chains

Student B – Trust Model & Validation Logic

Student B must individually:

- Write the complete validation algorithm in pseudocode (signature check on each link, issuer–subject matching, validity period, basic-constraints CA flag, key usage, name matching, revocation check, anchoring at a trusted root)
- Explain what each step protects against
- Explain why trust in the root cannot itself be verified cryptographically
- Explain how X.509 binds an identity to a public key and why that binding is what makes a signature meaningful

Student A – Failure-Case Experiments

Student A must individually:

- Design at least six failure cases: expired certificate, not-yet-valid certificate, wrong issuer, tampered certificate field, hostname mismatch, end-entity certificate used as a CA, revoked certificate
- Record which step of validation rejects each case
- Compare CRL and OCSP (freshness, size, privacy, availability) and show a case in which revocation is detected only if the check is actually performed
- Discuss what a "fail-open" revocation policy means for security

Joint Engineering Decision

Together determine:

If a CA is compromised or a certificate is mis-issued, what limits the damage, how long can the damage persist, and what would you change in a deployment to shorten that window?

Q5 – Kerberos: Authentication Without Sending Passwords (6 Marks)

Shared Group Work

Together:

- Study the Kerberos message flow (Authentication Server exchange, Ticket-Granting Server exchange, Client–Server exchange)
- Build a simulation with a client, KDC (AS + TGS) and one service, using symmetric encryption and authenticated tickets
- Include timestamps, ticket lifetimes, session keys and authenticators
- Test the normal flow and the attack/failure cases below

Student A – Protocol Design

Student A must individually:

- Draw the message-sequence diagram and label what is encrypted with which key
- Explain what the Ticket-Granting Ticket is for and why the user's long-term key is used only once per session
- Explain the purpose of the authenticator and the timestamp
- Explain how mutual authentication is achieved
- Identify which party can read which part of each message

Student B – Attack and Failure Evidence

Student B must individually:

- Demonstrate a replay of a captured authenticator with and without replay protection
- Demonstrate an expired ticket and a clock-skew failure
- Demonstrate what happens if the KDC is unavailable
- Record accept/reject outcomes and the reason for each
- Explain what a stolen ticket or an offline password-guessing attack on the AS reply would allow

Cross-Understanding Requirement

Student B must be able to explain:

Why does the client never send its password across the network, and what does the service learn about the client?

Student A must be able to explain:

What experimental evidence shows that the timestamp/authenticator mechanism actually prevents replay? This deliberately tests understanding across the assigned responsibilities.

Q6 – Advanced Security Mini-Investigations: RSA, DSA/DSS, ECDSA, ElGamal and Schnorr (7 Marks)

This question provides particularly clear individual evidence.

Student A – Hash Weakness and Signature Forgery

Student A takes primary responsibility for the collision-to-forgery investigation.

The student must:

- Explain the difference between a collision attack and a second pre-image attack
- Using your truncated-hash collision finder from Q1, build two different messages with the same short digest and show that a signature on one is valid for the other (use a signature scheme over the truncated digest)
- Explain why this attack requires the attacker to prepare both messages in advance
- Explain how a collision in MD5/SHA-1 enabled real-world forged certificates and what this means for X.509
- Explain the role of digest length and algorithm choice in deciding what a CA may use
- Recommend a hash policy for signatures and justify it

Student B Verification

Student B must provide a short explanation of:

Why does weakening the hash function weaken the signature, even when the signature algorithm itself (RSA/DSA) is unchanged?

Student B – Kerberos versus X.509 and Non-Repudiation

Student B takes primary responsibility for the framework comparison investigation.

The student must:

- Compare Kerberos and X.509/PKI in a table: key type, trusted third party, online/offline dependence, scalability, revocation, typical environment
- Explain which framework gives non-repudiation and which does not, and why (shared secret vs private key held by one party)
- Analyse the single point of failure of Kerberos and the trust-anchor risk in X.509
- Compare the number of key relationships required in a network of n users for pairwise shared keys, Kerberos with a KDC, and PKI
- Recommend which framework you would use for (i) a campus single sign-on system and (ii) signing legally binding electronic contracts

Student A Verification

Student A must provide a short explanation of:

Why can a Kerberos ticket prove who a user is to a service, yet not prove to a third party later that the user performed an action?

Thus, Q6 provides different primary tasks but mandatory cross-understanding.

Evidence Expected: worked toy example, verification trace, attack/failure cases and a comparison table covering RSA, DSA/DSS, ElGamal and Schnorr.

• Comparison requirement: Compare RSA, DSA/DSS, ElGamal and Schnorr signatures with respect to key material, randomness/nonce requirements, verification, forgery resistance and non-repudiation assumptions.

• Schnorr protocol: Describe the commitment–challenge–response sequence, derive the verification relation for toy parameters, and explain why the challenge must be unpredictable and why nonce reuse is dangerous.

• ElGamal signature scheme: Derive key generation, signing and verification for small toy parameters; test a valid signature and at least two altered-message/signature cases; explain how poor handling of signing randomness can weaken security.

Q7 – Design a Secure Authentication and Integrity Scheme from First Principles (10 Marks)

Problem Context – Secure Over-the-Air Update for Medical Devices

A manufacturer distributes firmware updates to thousands of infusion pumps in hospitals. Each pump must be sure that an update (a) has not been altered, (b) really comes from the manufacturer, and (c) the manufacturer cannot later deny having issued it. Pumps have limited memory, are often offline, and have unreliable clocks. Some pumps must also authenticate to the hospital's central server before accepting a service session.

Stage 1 – Independent Design Proposal

Before discussing the solution with each other:

Student A independently proposes:

- Which security goals each mechanism will address
- Which primitives (hash, MAC, signature, certificate, Kerberos-style ticket) will be used and where
- What key or certificate material each party holds
- A possible message flow for one update

Student B independently proposes:

- Which security goals each mechanism will address
- Which primitives (hash, MAC, signature, certificate, Kerberos-style ticket) will be used and where
- What key or certificate material each party holds
- A possible message flow for one update

Each student's initial proposal must be retained in the submission. No marks are deducted merely because an initial design is incorrect. The purpose is to provide evidence of independent design thinking.

Stage 2 – Compare the Two Designs

The pair must now compare the two independent proposals. Discuss:

- Where the designs agree?
- Where they differ?
- Strengths and weaknesses of each
- Missing threats or missing cases (rollback to an old version, stolen signing key, offline pump, wrong clock, revoked key)
- Which ideas should be retained?

Evidence Expected: A short design-comparison record showing how the two initial ideas influenced the final solution.

Stage 3 – Joint Final Design

Using the strongest ideas from the discussion, jointly develop the final scheme. The final design must identify:

- Threat model (who the attacker is and what the attacker can do)
- Security goals and the mechanism that achieves each
- Key and certificate management (issuance, storage, rotation, revocation)
- Message flow / protocol (diagram) for signing, distributing and verifying an update
- Behaviour when the pump is offline or its clock is wrong
- Complete pseudocode for signing and for verification

Stage 4 – Divide the Validation Responsibilities

Student A – Security Argument Validation

Student A must:

- State what each attack would need to succeed (e.g., 2^(n/2) work for a collision, forging a signature, compromising a key)
- Show, goal by goal, why the final design prevents or limits each attack
- Estimate the attacker's effort for your chosen digest and key sizes using the results from Q1 and Q3

- Identify the residual risks that remain after the design

Student B – Experimental Validation

Student B must:

- Design correctness and attack test cases (valid update, modified image, modified signature, replayed older version, expired or revoked certificate, wrong key)
- Implement and instrument the final verification procedure
- Compare with a straightforward baseline (e.g., a bare SHA-256 digest published beside the file) to show which attacks the baseline fails to stop
- Record accept/reject outcomes, verification cost (hash operations, signature verifications) and the size of the added metadata

Stage 5 – Cross-Defence

Student A must explain:

What experimental evidence supports the group's design?

Student B must explain:

How does the security argument follow from the group's choice of mechanisms, and what does each mechanism contribute?

This ensures that both students understand both the theoretical and experimental dimensions.

Stage 6 – Joint Evaluation

Together determine:

- Which of integrity, authentication and non-repudiation is guaranteed by which part of the design
- What costs the design introduces (verification time, certificate storage, key-management burden)
- Under what conditions the design could still fail (key compromise, flawed implementation, compromised root, bad clock)
- Under what conditions a simpler or a different mechanism (e.g., HMAC with shared key, Kerberos tickets) would be more appropriate